

AI 產力左移與 Spec-as-Source 工程框架： 以四維意圖規格體系與多層驗證鏈收斂生成式 AI 之隨機性

陳建維

凌網資訊・研發營運中心

Chris Chien-Wen Chen, R&D Operations Center, HyWeb Technology

Email: cwchen@hyweb.com.tw

摘要

生成式 AI (GenAI) 已快速進入軟體開發流程，然而將大型語言模型 (LLM) 直接生成交付程式碼，仍面臨三項結構性風險：Token 消耗不可控、機率取樣導致產出不一致，以及程式碼與業務需求間缺乏結構化追溯。本論文提出「AI 產力左移」方法論，主張將 AI 主要作用點由生命週期右端 (程式碼實作) 系統性移往左端 (需求探索與規格制定)。具體建構以「規格即源碼」為核心之 Spec-as-Source 五層工程框架：意圖層 (高階 Gherkin、DBML、haAPI、haPDL，並以 haARM 作為橫切面授權規格)、形式化層 (EBNF + JSON Schema + Z3)、中介表示層 (Domain IR、PDL as IL、Auth IR)、生成層 (三條確定性 codegen 管線)、驗證層 (多道漸進式驗證鏈)。本文以 AI 輔助生成之 23 份企業級縱向案例規格提供可行性實證，並從 8 回合整合除錯中萃取出五類反覆失敗模式分類學，同時揭露框架自身透過多層驗證鏈驅動橫切議題規格演進之自我修正能力。

關鍵字：AI 產力左移、規格即源碼、單一事實來源、形式化規格、中介表示層、多道驗證鏈、行為驅動開發

1. 緒論

1.1. 研究背景

生成式 AI 正深刻改變軟體開發實踐。GitHub Copilot 之受控研究顯示開發速度可提升約 55% [1]；Devin 進一步展示 AI 涵蓋全生命週期的潛力 [2]。然而將 LLM 直接投入交付程式碼生成面臨三項挑戰：成本不可控 (Token 消耗與生成量正相關，且 LLM 程式碼常隱含 Code Smells 與技術債 [3])；先天不確定性與表面正確性陷阱 (即便 temperature=0 仍存在非確定性 [4])；通過基本測試之程式碼仍可能有深層邏輯缺陷或邊界錯誤 [5]；需求追溯性斷裂 (開發者注意力由系統理解轉向提示詞優化，難以系統性評估變更影響 [6])。三項風險指向同一核心：若缺乏中介性結構化規格層，LLM 高熵輸出將直接暴露於交付層，持續放大維護、驗證與追溯成本。儘管 LLM 於需求工程已具基礎 [7], [19]，現有研究多將其定位為孤立對話輔助，未與下游程式碼生成管線結構化整合。

上述風險在需要高度一致性的企業級軟體產品線工程 (SPLE) [8] 情境下尤為致命。當多個事業群共用核

心平台時，若缺乏強制性的約束機制，極易因開發權分散而導致架構發散，最終使核心產品之演進跟不上市場節奏。若在未具備結構化護欄之情況下直接引入 LLM 生成程式碼，將以前所未有之速度加劇此類架構腐化。因此，如何建構一套既能釋放各團隊業務開發彈性、又能將核心架構收斂於中央管線之工程框架，成為本研究之核心工業動機。

1.2. 研究問題

- **RQ1**：如何重新定位 AI 在軟體生命週期中的角色，達成產能提升與隨機性可控之平衡？
- **RQ2**：如何設計一套規格體系作為單一事實來源，既承接 AI 輔助的需求探索產出，又有效約束下游自動化產出？
- **RQ3**：如何從規格體系自動導出驗證規格，使規格、測試與實作之間的一致性得以結構化維護？

1.3. 研究貢獻

本研究遵循 Wieringa 之設計科學研究 (Design Science Research, DSR) 方法論 [9]。針對 GenAI 於交付端之隨機性問題 (問題探究)，本文設計了 Spec-as-Source 工程框架 (處方設計)，並透過單一企業級縱向案例進行工程可行性與架構演進之實證 (處方驗證)。本研究旨在提供該設計知識於真實工業脈絡中之存在性證明，而非統計推論性之量化實證。本論文提出三類互不重疊之貢獻：

(C1) 概念貢獻：三段式 AI 產力左移模型，將 AI 介入層級 (Level 0 / 1 / 2) 與隨機性暴露範圍系統性對應，論證 Level 2 (規格層) 為較可控之 AI 作用點。

(C2) 架構與技術貢獻：Spec-as-Source 五層工程框架，含四維 SSoT (高階 Gherkin、DBML、haAPI、haPDL) 與橫切面授權規格 haARM；EBNF + JSON Schema + Z3 三重形式化機制；Domain IR / PDL as IL / Auth IR 中介表示層；三條確定性 codegen 管線；以及統合 AST Linter 與多道驗證鏈。礙於企業保密限制，實作程式碼未對外開源，然本文已詳盡揭露各 DSL 之三重形式化機制 (§3.4) 與核心一致性規則，確保研究架構之可追溯性與復現性。

(C3) 方法論與分類學貢獻：基於 23 份規格縱向工程案例，萃取出五大反覆失敗模式分類學；haARM v2

湧現之元結果——展示框架透過靜態分析揭露橫切議題盲區之自我修正能力。

2. AI 產力左移：理念與模型

2.1. 現行做法之結構性局限

當前「AI → Delivery Code」模式具兩項結構性問題。其一，隨機性放大效應：Copilot 所生成程式碼於風格一致性與錯誤處理上存在顯著不穩定性 [10]；若改以結構化規格（如 YAML DSL）為產出目標，隨機性可被限制於語法約束之有限狀態空間並由 Linter 即時攔截。其二，維護成本轉嫁：LLM 程式碼之可讀性與長期演化挑戰持續累積 [7]，反映出生成目標之抽象層級對系統可靠性具關鍵影響。

2.2. 三段式左移模型

本研究提出雙軸的三段式 AI 產力左移模型（圖 1）：

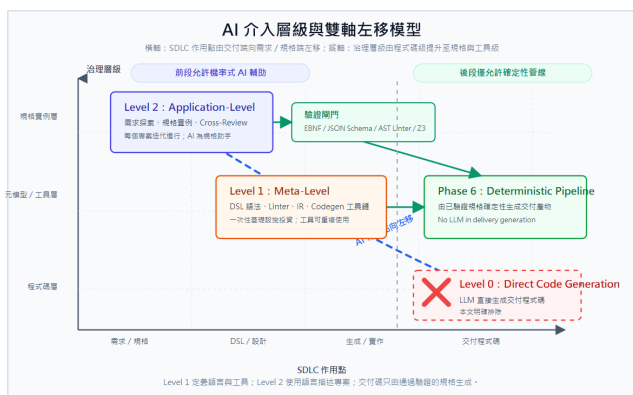


圖 1. AI 產力左移的雙軸模型。橫軸表示 AI 作用點由交付程式碼向需求與規格制定左移；縱軸表示治理層級由交付程式碼提升至 DSL 工具層與規格實例層。Level 1 為元級左移，AI 輔助 DSL、Linter 與 codegen 工具鏈的一次性設計；Level 2 為應用級左移，AI 輔助每個專案的需求探索與規格實例制定。本文明確排除 Level 0 直接生成交付程式碼；Phase 6 交付產物僅由通過 EBNF、JSON Schema、AST Linter、Z3 與人工審查之規格確定性生成。

- **Level 0**：AI 直接生成交付程式碼；本文排除。
- **Level 1**：AI 輔助 DSL 語法、Linter、IR 與 codegen 工具鏈設計；屬框架基礎設施的一次性建置。
- **Level 2**：AI 輔助需求探索、規格實例制定與 cross-review；屬每個專案反覆執行的規格工作。

三段式模型之維度比較如表 1 所示。

2.3. AI 在 WA-RAPTOR 中的角色界定

WA-RAPTOR 是本研究實現上述模型之規格驅動開發框架，AI 角色限定於 Level 1 與 Level 2，並透過三類機制參與：(i) **Discovery Agent** 以結構化提示掃描欄位引用完整性與跨維度一致性，輸出釐清問題清單；(ii) **Clarify-and-Translate Agent** 以對話式

few-shot prompting 協助領域專家逐項澄清，並將自然語言回答轉譯為規格更新操作；(iii) **Cross-Review Mechanism** 對重要架構決策進行多 LLM 獨立分析 (Opus / Gemini / GPT 對抗性 prompts)，由人類整合者裁決。本設計呼應 Specification by Example [11]，並引入 human-in-the-loop 以保留領域專家對規格之最終決策權；所有交付層程式碼均由經驗證之規格透過確定性 codegen 產出，不採 Level 0 模式。

3. Spec-as-Source 工程框架

3.1. 設計理念

「規格即源碼」(Spec-as-Source) 之核心主張：專案 SSoT 為結構化規格而非程式碼；程式碼為規格在固定生成流程下之確定性衍生物，當規格變更時透過自動化流程同步更新。本研究承襲 MDE「模型為核心」思想 [12]，但採純文字 YAML/DBML/Gherkin 作為規格語言 DSL [13] 取代圖形化 UML，帶來三項工程優勢：與 Git 原生整合、易於 LLM 解析生成、提升非技術利害關係人理解性；並透過限制生成程式碼為唯讀以避免 round-trip engineering 之複雜性 [14]。本研究之工程化實踐進一步顯示：單純的 DSL + 模板展開不足以支撐企業規模 spec-driven codegen，真正的可信度來自分層之工程框架（圖 2）。

3.2. 五層工程框架

本框架之核心架構由五個層次構成（見圖 2）：意圖層（釐清業務與系統邊界）、形式化層（透過嚴格約束確保語意正確）、中介表示層（統合跨層推導邏輯）、生成層（執行單向確定性轉換），以及驗證層（確保最終產出與原始規格對齊）。此分層設計有效地將高熵之非結構化需求，逐步收斂為低熵之機器可執行指令。



圖 2. Spec-as-Source 五層工程框架。Layer 1–3 確定「什麼是規格」；Layer 4 將規格確定性地轉為程式碼；Layer 5 透過多層驗證鏈確認結果在編譯、執行、整合三階段均對齊規格意圖。

3.3. 意圖層 (Intent Layer)：四維意圖與橫切面

意圖層規格體系如表 2 所示。

表 1. 三段式左移模型之維度比較

維度	Level 0	Level 1	Level 2
AI 作用點	交付程式碼	DSL / Linter / Codegen 工具鏈	需求探索 / 規格實例
治理層級	程式碼級	元級 / 工具級	應用規格級
是否一次性	否，隨需求反覆生成	是，框架基礎設施投資	否，每個專案迭代進行
隨機性暴露範圍	最大，直接污染交付碼	受限於工具設計期	受限於可審查、可驗證規格
主要護欄	Code review / 測試	DSL 設計審查、Linter、codegen 測試	EBNF、JSON Schema、AST Linter、Z3、人工審查

表 2. 意圖層規格體系

維度	規格語言	職責
業務意圖	高階 Gherkin	使用者故事與驗收條件
領域模型	DBML	實體、屬性、關係、約束（主節點樹）
服務意圖	haAPI	後端 API 能力抽象描述
頁面意圖	haPDL	前端頁面呈現抽象描述
橫切：授權	haARM	Actor / Resource / Permission (含 Scope 與 Condition) 之正規化定義；被各層共同引用

haARM 獨立作為橫切面規格，為本研究於工程實踐中之關鍵架構發現：若將授權邏輯以「角色白名單」形式散落於各 DSL 中，隨著系統規模擴大，極易產生跨 DSL 之語意漂移（詳見 §5.4 案例）。因此，本框架強制將帶有多維結構（Resource × Action × Scope × Condition）的權限模型抽取為獨立規格，確保全域授權邏輯之正交性與單一事實來源。

3.4. 形式化層：規格三重形式化機制

每個 DSL 均具備語法、結構與語意之三層形式化機制：(i) **EBNF** 文法確立語法邊界與解析樹；(ii) **JSON Schema** 作為設計階段 Linter 與 IDE 自動補齊之機器可讀契約；(iii) **Z3** 約束 [15] 透過定理證明器深度驗證跨欄位、跨 DSL 之語意一致性與邏輯衝突。實測規模：haPDL 之 EBNF 250+ 行、JSON Schema 600+ 行（含 11 語義表）、Z3 涵蓋導覽一致性與與頁面可達性；haAPI 之 EBNF 320+ 行、JSON Schema 700+ 行、Z3 驗證型別一致性、端點唯一性與授權邊界衝突；haARM 之 BNF 195 行、5 區段 schema、Z3 則驗證職責分離 (SoD) 衝突、基數約束與循環參照；例如，框架會將 AST 解析為一階邏輯公式（如 `Implies(requiresRoles(op), requiresAuthentication(op))`），交由 SMT 求解器靜態攔截潛在的授權邊界矛盾。三重機制使規格由非結構化描述轉化為機器可驗證契約，構成多階段驗證鏈成立之必要前提。

3.5. 統合 AST Linter

四維意圖規格與 haARM 之內部一致性依賴一套統合抽象語法樹所構建之 Linter。分析器將 DBML 解析為主節點樹（實體與欄位定義），將 haAPI、haPDL、haARM 解析為關聯參照樹；當分析規則於主節點樹找

不到參照樹所引用之節點，即在編輯時拋出一致性錯誤。Linter 核心一致性規則列於表 3。

表 3. LINTER 核心一致性規則

規則 ID	規則描述	嚴重度
L-01	haAPI.api_for 實體須存在於 DBML	Error
L-02	haPDL.entity 實體須存在於 DBML	Error
L-03	haPDL.columns 欄位（去符號）須存在於對應 DBML 實體	Error
L-04	haAPI.filterable_by 欄位須存在於對應 DBML 實體	Error
L-05	高階 Gherkin Scenario 須至少映射一個 haAPI endpoint	Warning
L-06	haAPI.access.roles 角色須與 haARM actors.role 一致	Error
L-07	haAPI.access.required_permissions 須引用 haARM 已定義之 permission.id	Error
L-08	haPDL.security.permission_refs 須引用 haARM 已定義之 permission.id	Error

L-07 / L-08 為 haARM 加入後新增之規則，直接作用於 §5.4 揭露之授權語彙漂移問題。表 3 僅列出具跨維度驗證指標意義之核心規則。於實際工程實作中，該 Linter 另包含 40+ 項涵蓋型別推論衝突與生命週期狀態機之基礎校驗規則。

3.6. 中介表示法 (IR) 與中介語言 (IL)

透過實作三組獨立 codegen 管線，本研究歸納出核心架構守則：避免將 **DSL** 直接透過模板展開為目標程式碼，否則極易導致多生成端因各自推論而產生「契約漂移」；引入顯式 **IR** 或 **IL**，乃維持下游全局一致性之關鍵。三項實踐：(1) **whyAPI** 採「Domain IR + 各語言渲染器」解耦設計，杜絕 Controller 與 Service 因各自從規格推導函式簽章而產生之介面不一致；(2) **haPDL2PDL** 將 PDL 提升為 IL，透過六階段解析管線（上下文綁定 → 預設值注入 → 慣例套用 → DBML 關聯 → haAPI 映射 → 覆寫機制）將高階意圖確定性展開；(3) **haARM** 將原本隱晦字串權限升級為 Resource × Action × Scope × Condition 之強型別 Auth IR。共通之「執行力優先 **IR**」原則：IR 結構必須由下游強制執行端需求主導，而非由上游投影端彈性需求決定。

3.7. 三條衍生管線

- 後端：haAPI + DBML → Domain IR → Type-Spec → Java Spring Boot

- 前端：haPDL + DBML → PDL (IL) → Vue 3 元件
- 驗證：haAPI + haPDL + 高階 Gherkin → 低階 Gherkin → 自動化測試

其中，驗證管線「由 GWT (Given-When-Then) 語句直接衍生可執行測試 (Step Functions)」之自動化概念，係啟發自業界專有框架 SpecFormula [16]。此管線所導出之「低階 Gherkin」，其抽象層級精準介於業務導向之高階 Gherkin 與底層測試斷言之間，從結構上確保了測試程式碼與規格之同步。此外，授權機制作為延伸形成第四條輔助管線：haARM → Auth IR → Mock Actor / JWT Fixture → 安全情境測試。

4. 流程與一致性保證

4.1. 七階段 AI 協同流程

七階段需求發展流程列於表 4。

七階段對應 BDD 之 Discovery → Specification → Automation 轉換 [17]。Phase 3 為發散與收斂交界，是容許最高 AI 隨機性之階段；Phase 5 則作為安全護欄，透過 Linter、Z3 與多層驗證鏈對 Phase 4 草案進行嚴格自動化檢查；Phase 6 完全排除 AI 介入。

4.2. 多階段驗證鏈

漸進式失效捕獲原則：靜態語法通過不代表業務語意正確；後端編譯成功不代表 Spring 容器能順利啟動；EJS 模板渲染無誤不代表 Vue 框架能正確解析虛擬 DOM。本研究設計六道驗證關卡：(1) 靜態規格驗證——EBNF、JSON Schema、Linter 與 Z3；(2) 生成器單元測試；(3) 靜態編譯 (mvn package)；(4) 服務啟動 (Spring Boot IoC)；(5) 執行期 API 驗證；(6) 前端整合與冒煙測試。實證觀察顯示 23 類生成器缺陷皆具「跨層逃逸」特性，例如帶雙引號之 `hasPermission(["admin"])` 通過 Stage 2 模板渲染卻於 Stage 6 因 HTML 屬性引號衝突靜默失敗，印證跨層 N 道驗證鏈之必要性。

5. 實證評估：縱向案例研究

章節邊界：本章透過企業級後台系統（專案代號 aisystem）之導入，為 §3 框架提供第一手工程可行性 (Feasibility) 實證，並歸納出具分類學價值之失敗模式與驅動規格演進之元結果。

5.1. 案例設定與工程結果

aisystem 涵蓋使用者管理、組織架構、權限角色與領域知識庫等模組。本研究將既有 aisystem-api (Java Spring Boot) 與 aisystem-main (Vue 3) 作為端到端驗證宿主，並由 23 份 haAPI (共 24 份，1 份未投入量產)、20 份 haPDL 與 22 個領域實體之規格透過多層驗證鏈自動生成 API 控制器與 UI 元件，與既存手寫程式碼共存且無縫整合。

端到端生成與整合測試結果：23 份 haAPI 規格全數通過批次編譯，成功生成 369 個 Java 檔

案（涵蓋 303 個 Entity/DTO 及各 22 個 Repository/Service/Controller）。後端管線之 185 項核心迴歸測試皆通過，專案建置成功 (BUILD SUCCESS) 且 Spring Boot 容器啟動僅耗時 7.4 秒。於執行期驗證中（以 `/api/info-user` 為例），CRUD 操作與 JWT 權限管控皆如預期運作。前端管線則通過全部 31 項測項（含 21 項單元測試、6 項端對端測試與 4 項黃金快照），如 UserList 等複雜頁面之渲染與互動均測試無誤。上述所有自動化產出皆具備至原始規格之完整追溯性。

5.2. 變更影響：跨層命名收斂之 SSoT 案例

為驗證 SSoT 架構在系統演進期之維護效益，本研究記錄了一次真實的跨層命名收斂任務——將所有 GET list 端點之回應欄位由 `doc_list` 統一更名為 `data_list`。在傳統手寫架構中，此類變更極易因前後端開發者「各自推斷契約」而遺漏，進而引發執行期資料綁定錯誤（即 §5.3 探討之模式 A：契約漂移）。

然而在本框架下，開發者僅需於生成器核心之 IR 轉換邏輯（3 份 TS 模組，共 10 處 AST 修改）進行單一變更，即可精準驅動整個系統之自動化對齊：後端 23 份規格之 Java DTO 與 Service 介面全量重生；前端 EJS 模板與數十個 Vue 頁面完成同步綁定；就連 4 個前端黃金快照亦由 CI 流程自動觸發更新。此案例具體量化了 Spec-as-Source 的槓桿效應——透過中介表示層 (IR) 作為唯一防護，將原本高風險的跨層重構轉化為可預測的確定性衍生操作。

5.3. 規格驅動生成之反覆失敗模式分類學

對 8 回合端到端整合除錯所修復之 23 類 codegen 缺陷，以開放式編碼 (Open Coding) 依據「失敗觸發層」與「根因歸屬層」兩個維度進行歸納，提煉出 5 個跨層管線之根失敗模式（表 5）；因本研究為單一研究者執行，分類之穩健性待後續多人編碼驗證。

此分類揭露了 AI 輔助開發之一項重要洞見：由於 LLM 具備高速且高複雜度的規格生成能力，其產出會對底層生成管線與宿主框架的「隱性規約」進行嚴苛的壓力測試。這使得傳統手寫開發中可被工程師直覺避開的邊界衝突（如命名空間重疊、框架潛規則），在 AI 驅動下被顯著放大。

5.4. haARM 湧現之後設研究

在全量生成過程中，本研究發現前端模板出現 `hasPermission(['admin'])` 之語意錯置。逆向追溯揭露，由於初期框架遺漏將「授權」此橫切關注點獨立規範化，導致「角色 ID」被錯認為「權限 ID」，並以 `string[]` 弱型別連續穿透五層管線。此歷史包袱導致型別系統完全喪失對權限語意的邊界區分能力。

此架構矛盾直接驅動了 haARM 之重構：將字串權限升級為涵蓋 `Resource × Action × Scope × Condition` 之強型別結構，並擴充對應 Linter 規則。此後設觀察證實了 Spec-driven 框架具備自我修正能力：當生成管線之深廣度觸碰未嚴謹定義之橫切議題時，多層驗證機制會以型別衝突強制將潛藏的架構技術債浮出檯面，進而迫使規格演進。

表 4. 七階段需求發展流程

階段	名稱	主要產出	AI 角色
Phase 1	業務與意圖探索	User Journey & 高階 Gherkin	結構化整理與語意建議
Phase 2	領域建模	DBML	實體萃取輔助
Phase 3	需求澄清與收斂	釐清模糊邊界與邏輯衝突	Discovery + Clarify Agent
Phase 4	規格形式化	haAPI + haPDL + haARM 草案	結構化規格草案生成
Phase 5	多層驗證與導出	低階 Gherkin + 靜態分析報告	驅動 Linter 與 Z3
Phase 6	確定性生成	後端 API + 前端 UI 元件	無 AI；純確定性 Codegen
Phase 7	迭代與影響分析	規格版本更新	變更影響範圍評估

表 5. 規格驅動程式碼生成之失敗模式分類學

模式	模式名稱	症狀與根因	防禦對策 (Linter/設計)
A	契約漂移	症狀：不同生成端推導出微差異簽章。 根因：多端各自推斷語意，缺乏唯一真理。	契約解析上抽至 IR 層，確保單一推導來源。
B	命名空間碰撞	症狀：同作用域重複宣告變數（如 <code>qSql</code> ）。 根因：生成器錯誤假設「同類邏輯僅出現一次」。	實作作用域變數追蹤並動態附加 Unique ID。
C	型別三角不一致	症狀：規格 <code>int</code> → DB <code>BigDecimal</code> → 拋出例外。 根因：IR、DB 與宿主語言之型別系統相互獨立。	建立顯式跨系統型別映射矩陣，編譯期檢查。
D	框架隱性規約衝突	症狀：Spring JPA 錯誤解析；複合主鍵未宣告致崩潰。 根因：宿主框架存在「靜態通過、執行期觸發」之潛規則。	將隱性規約顯式化為生成前 Linter 規則。
E	反模式繼承	症狀：新生成 DTO 錯誤繼承既有肥大實體。 根因：與龐大 legacy 系統並存時繼承舊架構壞味道。	導入「反模式偵測器」，特定場景觸發降級改寫。

6. 討論

6.1. 與相關工作之差異

本文與既有範式之主要差異整理於表 6。

綜合新穎性：既有文獻多將上述範式視為獨立之研究軌道，導致「AI 產出物（具機率與隨機性）」與「系統交付要求（需絕對確定性）」之間，始終存在難以跨越的信任鴻溝。本研究之核心原創性不在於發明單一底層技術，而在於架構層面之系統性整合——首創將 AI 需求澄清、多維 SSoT、中介表示層（IR）與多段驗證鏈鑄為一條端到端管線。此設計成功將原本孤立之研究領域，轉化為一套「以 AI 擴展需求探索邊界、以規格收斂實作確定性」之軟體工程新範式。

6.2. 適用邊界與效度威脅

本框架當前之四大效度邊界與應對策略如下。內部效度：操作示例由作者執行，難排除確認偏誤；後續研究將招募未接觸本框架之獨立受試者。建構效度：單純以規格行數與工件數為效率代理，恐無法完整反映工程師之認知負荷，故後續研究將以多元量化指標進行三角驗證以降低偏誤。外部效度：本實證強烈依賴 CRUD-intensive 之企業後台情境，其結論不宜直接外推至高度動態互動、低延遲即時系統或演算法密集型應用。結論效度：當前案例已提供框架建構可行性之存在性證明，統計層面之因果顯著性主張則留待後續受控實驗予以確立。

6.3. 理論啟示：從引用一致性邁向語意對齊

規格驗證機制的演進，必須跨越「引用一致性」的表象，深入處理跨 DSL 的「語意對齊」。早期的 Linter

僅檢查實體或欄位於主節點樹的存在性（語法與結構層次）；然而 §5.4 案例揭示，即便所有參照皆合法，系統仍可能發生語意層錯置（如角色與權限混用）。此現象精準呼應了需求工程與驗證測試（RE-VV）間長期存在的對齊挑戰 [20]：系統至多只能被證明「相對於規格」為正確，卻無法用傳統驗證機制保證「規格本身」的語意正確性。本研究指出，Spec-as-Source 的下一階段演進，在於引入具備強語意表達能力之「型別化中介表示」——正如本研究將隱晦的字串權限，重構為受 Z3 嚴格約束的 Auth IR，這正是將橫切語意轉化為機器可驗證契約之具體實踐。

6.4. 工程代價與 AI Agent 之互補性

採行 Level 2 左移並非毫無代價，其涉及四項工程挑戰：(1) 基礎設施建置：DSL 與管線需深度結合 DDD，顯著提高團隊初期認知負荷；(2) 領域語彙治理：需仰賴多模型交叉審查以收斂語意衝突；(3) 隱性規約顯式化：需將宿主框架（如 Spring/Vue）之潛規則逐一轉化為 Linter 規則；(4) 架構機會成本：強制執行之 IR 將限縮前端開發之自由度。

其投資報酬率高度取決於專案屬性：對單一短期 PoC 而言 ROI 往往為負；但若將本框架作為軟體產品線（SPLE）之核心生成引擎，支援跨事業群每年數十個專案交付，前期建置成本將被規模化之維護紅利所吸收。此外，本研究主張 Level 2 與純 AI Coding Agent 並非互斥，而應走向「先期約束、後端賦能」——以四維 SSoT 作為業務意圖之絕對護欄，再將 AI Agent 定位為下游之受控實作者。當未來 Agent 生成之可解釋性與確定性達標時，即可將其無縫整合為本框架內受規格驅動之高彈性動態管線。

表 6. 與既有範式之主要差異

範式	既有缺口	本文貢獻
MDE [12], [14]	UML 與 LLM 互動有摩擦；round-trip 同步障礙	純文字 YAML/DBML/Gherkin；嚴格單向確定性生成
Low-Code 平台 [18]	黑箱執行環境致 vendor lock-in	開放 DSL 與可檢視 codegen，產出可獨立維護
BDD/SbE [11], [17]	低階 Gherkin 仰賴人工同步	自動推導使測試成為規格之衍生物
AI4RE [7], [19]	LLM 多停留於對話助手，未銜接下游管線	AI 產出直接形式化為機器可驗證規格
AI Coding Agent [2], [5], [10]	LLM 機率本質與幻覺風險	將 AI 隨機性收斂於規格層；Level 0 與 Level 2 互補 (§6.4)
SPLE [8]	核心資產與衍生專案之變異管理仰賴手動特徵模型	以四維 SSoT 作為核心資產之機器可驗證規格；codegen 管線確保衍生專案之架構一致性

7. 結論

本論文提出 AI 產力左移方法論與 Spec-as-Source 五層工程框架，主張將 GenAI 能力由末端程式碼交付，系統性向左推移至結構化規格制定。透過將 AI 之機率性限縮於受控的語法空間，本框架旨在提升開發流程之確定性與端到端追溯性。針對本文提出之三大研究問題，得出以下初步實證支持：對 **RQ1** (AI 角色邊界)，三段式左移模型確立了 Level 2 為較佳之工程可控點，並由 §5 縱向案例驗證其穩定性；對 **RQ2** (意圖防護機制)，四維 SSoT 搭配 haARM 與統一 AST 構築了可機器強制檢驗的防護網，成功驅動跨層語意對齊 (§5.4)；對 **RQ3** (規格與測試同構)，由 SSoT 確定性導出低階 Gherkin 之機制，從結構上顯著約束了測試脫節風險，23 份規格全數通過端到端測試即為建構性覆蓋之初步實證。

關鍵發現：基於嚴謹規格之框架，其 Linter 與整合測試機制成功揭露了未被建模之橫切議題（授權語彙漂移），進而驅動 haARM v1 → v2 之架構演進。此種由驗證機制觸發之「結構化回饋循環」，構成 Spec-driven 框架相較於純 AI Coding Agent 之核心工程優勢。

未來工作：(1) 量化驗證：針對 Level 0 與 Level 2 之穩定性、維護性與追溯性差異，設計並執行多受試者受控實驗；(2) 開放基準測試：計畫提煉一套去識別化之合成規格測試集與標準化評估腳本，為社群提供可公開驗證之量化尺規；(3) 多橫切面正交疊加之工程可行性，並延伸至軟體產品線工程 (SPLE)；(4) 探索強型別 IR 與多道驗證鏈如何約束 AI Coding Agent 之生成幻覺，建構「以確定性管線為基底、以機率性代理為擴充」之混合式工程架構。

參考文獻

- [1] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirel, "The impact of AI on developer productivity: Evidence from GitHub Copilot," *arXiv preprint arXiv:2302.06590*, 2023.
- [2] Cognition AI, "Introducing Devin, the first AI software engineer," *Cognition Blog*, Mar. 2024.
- [3] B. Yetiştiren, I. Özsoy, M. Ayerdem, and E. Tüzün, "Evaluating the code quality of AI-assisted code generation tools: An empirical study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT," *Journal of Systems and Software*, vol. 207, p. 111868, Jan. 2024.
- [4] S. Ouyang, J. M. Zhang, M. Harman, and M. Wang, "An empirical study of the non-determinism of ChatGPT in code generation," *arXiv preprint arXiv:2308.02828*, 2023.
- [5] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation," in *Proc. NeurIPS*, 2023.
- [6] D. Russo, "Navigating the complexity of generative AI adoption in software engineering," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 5, art. 135, pp. 1–50, 2024.
- [7] A. Fan et al., "Large language models for software engineering: Survey and open problems," in *Proc. IEEE/ACM ICSE-FoSE*, 2023, pp. 31–53.
- [8] K. Pohl, G. Böckle, and F. J. van der Linden, *Software Product Line Engineering: Foundations, Principles and Techniques*. Berlin: Springer, 2005.
- [9] R. J. Wieringa, *Design Science Methodology for Information Systems and Software Engineering*. Berlin: Springer, 2014.
- [10] A. N. Dakhel et al., "GitHub Copilot AI pair programmer: Asset or liability?," *Journal of Systems and Software*, vol. 203, p. 111734, Sep. 2023.
- [11] G. Adzic, *Specification by Example: How Successful Teams Deliver the Right Software*. Manning, 2011.
- [12] M. Brambilla, J. Cabot, and M. Wimmer, *Model-Driven Software Engineering in Practice*, 2nd ed. Morgan & Claypool, 2017.
- [13] M. Voelter, *DSL Engineering: Designing, Implementing and Using Domain-Specific Languages*. Createspace Independent Publishing Platform, 2013.
- [14] S. Sendall and W. Kozaczynski, "Model transformation: The heart and soul of model-driven software development," *IEEE Software*, vol. 20, no. 5, pp. 42–45, Sep. 2003.
- [15] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Proc. TACAS (LNCS 4963)*, Springer, 2008, pp. 337–340.
- [16] Waterball Software Academy, "SpecFormula Dev Framework and ISA-Level Gherkin," unpublished internal course material, 2026.
- [17] J. F. Smart and J. Molak, *BDD in Action*, 2nd ed. Manning, 2023.
- [18] D. Di Ruscio et al., "Low-code development and model-driven engineering: Two sides of the same coin?," *Software and Systems Modeling*, vol. 21, no. 2, pp. 437–446, Apr. 2022.
- [19] H. Cheng et al., "Generative AI for requirements engineering: A systematic literature review," *Softw. Pract. Exper.*, 2025.
- [20] E. Bjarnason et al., "Challenges and practices in aligning requirements with verification and validation: a case study of six companies," *Empirical Software Engineering*, vol. 19, no. 6, pp. 1809–1855, 2013.